

## Chapter 3

### Storage Management

#### Memory Management

Memory management is the functionality of an operating system which handles or manages primary memory and moves processes back and forth between main memory and disk during execution.

Memory management keeps track of each and every memory location, regardless of either it is allocated to some process or it is free. It checks how much memory is to be allocated to processes. It decides which process will get memory at what time. It tracks whenever some memory gets freed or unallocated and correspondingly it updates the status. The part of OS that manages the memory hierarchy is called memory manager.

- Keeps track of used/unused part of memory.
- Allocate/de-allocate memory to processes.
- Manages swapping between main memory and disk.
- Protection and sharing of memory

#### Logical and Physical Memory

**Logical memory** enables the user to use large amount of memory to store data. It defines the way to organize the physical memory such as RAM and cache. This enables the Operating System to arrange memory into a logical manner such as assigning a logical address.

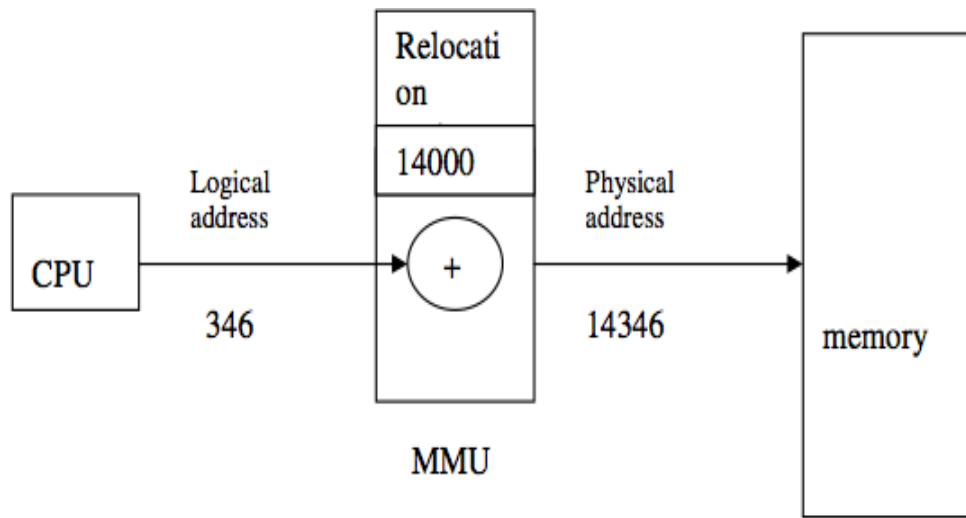
**Physical memory** refers to the actual RAM of the system, which usually takes the form of cards attached onto the motherboard. Also called primary memory, it is the only storage type directly accessible to the CPU and holds the instructions of programs to execute. Physical memory is linearly addressable; memory addresses increase in a linear fashion and each byte is directly addressable.

#### Logical address

An address generated by the CPU is commonly known as Logical address. When we give the problem to the computer then our computer passes the problem to the processor through logical address, which is not seen, this address called logical address. And the set of all logical addresses generated by a program is known as logical address Space.

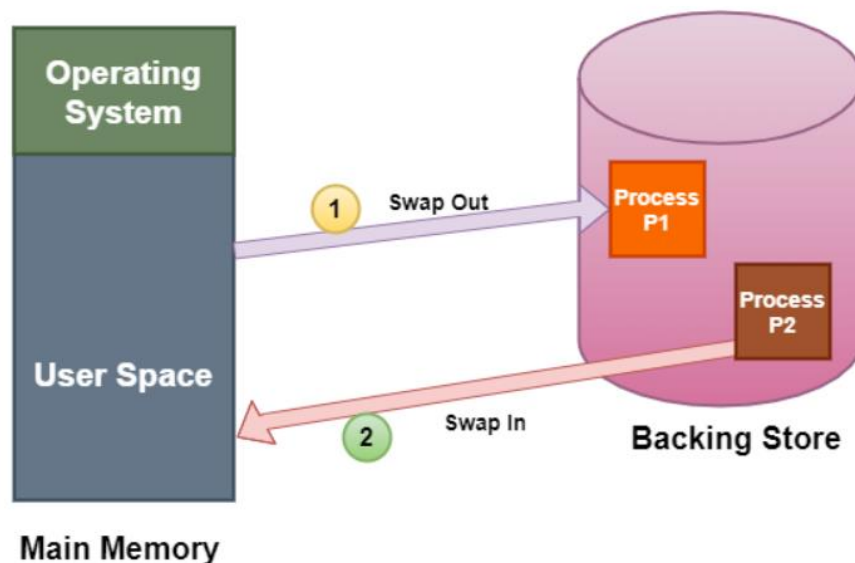
#### Physical address

When our processor create process and solve our problem then we store data in primary memory through address called physical address. In simple word the address seen by the memory unit that is one loaded into the memory address register of the memory is commonly refereed as the Physical Address. And the set of all physical addresses corresponding to these logical addresses is physical address space.



## Swapping

- Swapping is mechanisms in which a process can be swapped temporarily out of main memory (or move) to secondary storage (disk) and make that memory available to other processes.
- Backing store is a usually a hard disk drive or any other secondary storage which is fast in access and large enough to accommodate copies of all memory images for all users. It must be capable of providing direct access to these memory images.
- At some later time, the system swaps back the process from the secondary storage to main memory.

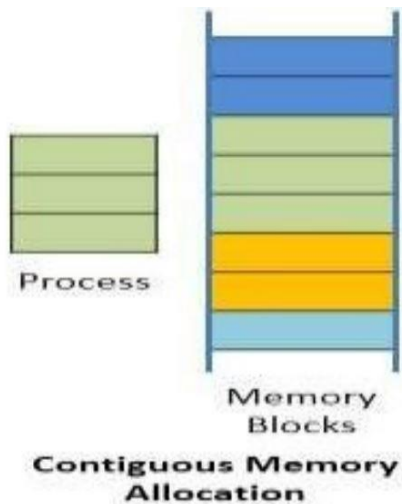


## Storage Allocation

The process of allocating the block of memory to a process is known as storage allocation. Memory is a large array of bytes, where each byte has its own address. The memory allocation can be classified into two methods:

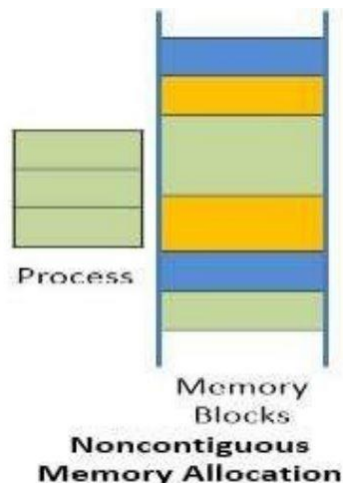
## I. Contiguous Memory Allocation

The Contiguous memory allocation is one of the methods of memory allocation. In contiguous memory allocation, when a process requests for the memory, a single contiguous section of memory blocks is assigned to the process according to its requirement. The contiguous memory allocation can be achieved by dividing the memory into the fixed- sized partition and allocate each partition to a single process only. The contiguous memory allocation also leads to the internal fragmentation. Like, if a fixed sized memory block allocated to a process is slightly larger than its requirement then the left-over memory space in the block is called internal fragmentation. When the process residing in the partition terminates the partition becomes available for another process.



## ii. Non-contiguous memory

The Non-contiguous memory allocation allows a process to acquire the several memory blocks at the different location in the memory according to its requirement. The non-contiguous memory allocation also reduces the memory wastage caused due to internal and external fragmentation. As it utilizes the memory holes, created during internal and external fragmentation. The non-contiguous memory allocation has an advantage of reducing memory wastage but, it increases the overheads of address translation. As the parts of the processes are placed in a different location in memory, it slows the execution of the memory because time is consumed in address translation.

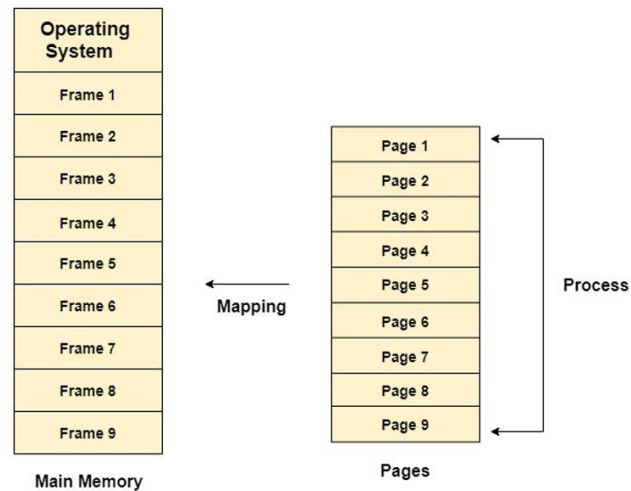


## PAGING

- Paging is a memory management scheme by which a computer stores and retrieves data from secondary storage for use in main memory.
- Paging is a process of reading data from, and writing data to the secondary storage.
- Partition memory into small equal fixed-size chunks and divide each process into the same size chunks.
- The chunks of a process are called pages.
- The chunks of memory are called frames.
- The basic idea behind paging is to only keep those part of the processes in memory that are actually being used
- Paging is non-contiguous memory allocation technique.
- The basic idea behind paging is to only keep those part of the processes in memory that are actually being used
- Paging:
  - ❖ divides processes to a fixed sized page
  - ❖ Then selectively allocates pages to frames in memory, and
  - ❖ manages (moves, removes, reallocates) pages in memory
- The logical memory of the process is still contiguous but the pages need not be allocated contiguously in memory
- By dividing the memory into fixed sized frames, we can eliminate external fragmentation • However, paging does not eliminate internal fragmentation
- Summary: breaking process memory into fixed sized pages and map them to a physical frame of memory OS needs to do two things
  1. Mapping the pages to frame, maintain a page table
  2. Translate the virtual address (contiguous) into physical address
  3. Address generated by CPU (logical address) is divided into:
    - Page number  $p$  is an index into a page table that contains base address of each page in physical memory.
    - Page offset  $d$  is a displacement, combined with base address to define the physical memory address that is sent to the memory unit.

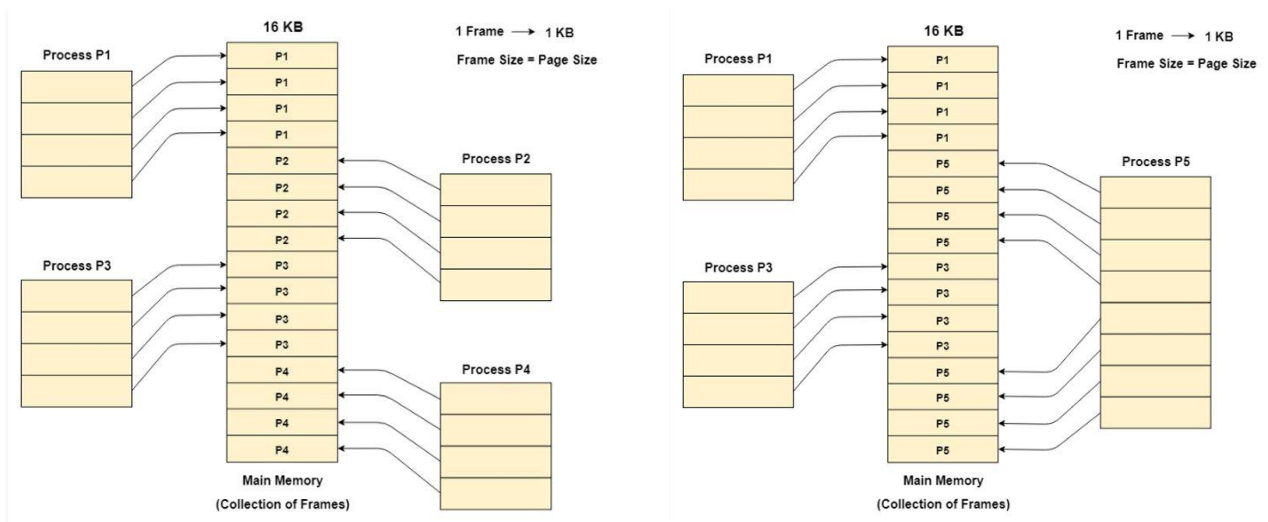
In Operating Systems, Paging is a storage mechanism used to retrieve processes from the secondary storage into the main memory in the form of pages. The main idea behind the paging is to divide each process in the form of pages. The main memory will also be divided in the form of frames. One page of

the process is to be stored in one of the frames of the memory. The pages can be stored at the different locations of the memory. Pages of the process are brought into the main memory only when they are required otherwise, they reside in the secondary storage. Different operating system defines different frame sizes. The sizes of each frame must be equal. Considering the fact that the pages are mapped to the frames in Paging, page size needs to be as same as frame size.



### Example

Let us consider the main memory size 16 Kb and Frame size is 1 KB therefore the main memory will be divided into the collection of 16 frames of 1 KB each. There are 4 processes in the system that is P1, P2, P3 and P4 of 4 KB each. Each process is divided into pages of 1 KB each so that one page can be stored in one frame. Initially, all the frames are empty therefore pages of the processes will get stored in the contiguous way. Frames, pages and the mapping between the two are shown in the image below.



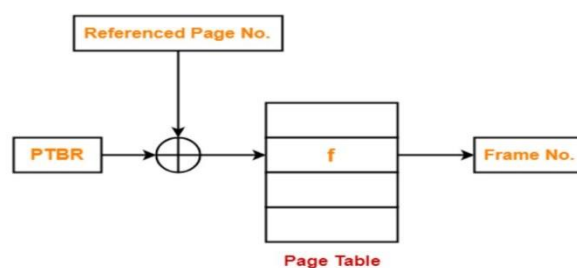
Let us consider that, P2 and P4 are moved to waiting state after some time. Now, 8 frames become empty and therefore other pages can be loaded in that empty place. The process P5 of size 8 KB (8

pages) is waiting to get inside the ready queue. Given the fact that, we have 8 non-contiguous frames available in the memory and paging provides the flexibility of storing the process at the different places. Therefore, we can load the pages of process P5 in the place of P2 and P4.

## Page table

A page table is the data structure used by a virtual memory system in a computer operating system to store the mapping between virtual addresses and physical addresses.

- Page table is stored in the main memory.
- Number of entries in a page table = Number of pages in which the process is divided.
- Page Table Base Register (PTBR) contains the base address of page table.
- Each process has its own independent page table.



Obtaining Frame Number Using Page Table

- A page table entry contains several informations about the page.
- The information contained in the page table entry varies from operating system to operating system.
- The most important information in a page table entry is frame number.



Page Table Entry Format

### 1. Frame Number-

- Frame number specifies the frame where the page is stored in the main memory.
- The number of bits in frame number depends on the number of frames in the main memory. 2.

### 2. Present / Absent Bit-

- This bit is also sometimes called as valid / invalid bit.
- This bit specifies whether that page is present in the main memory or not.
- If the page is not present in the main memory, then this bit is set to 0 otherwise set to 1. 3.

### 3. Protection Bit-

- This bit is also sometimes called as "Read / Write bit".

- This bit is concerned with the page protection.
- It specifies the permission to perform read and write operation on the page.
- If only read operation is allowed to be performed and no writing is allowed, then this bit is set to 0.
- If both read and write operation is allowed to be performed, then this bit is set to 1. 4.

#### **4. Reference Bit-**

- Reference bit specifies whether that page has been referenced in the last clock cycle or not.
- If the page has been referenced recently, then this bit is set to 1 otherwise set to 0.
- Reference bit is useful for page replacement policy.
- A page that has not been referenced recently is considered a good candidate for page replacement in LRU page replacement policy.

#### **5. Caching Enabled / Disabled-**

- This bit enables or disables the caching of page.
- Whenever freshness in the data is required, then caching is disabled using this bit.
- If caching of the page is disabled, then this bit is set to 1 otherwise set to 0.

#### **6. Dirty Bit-**

- This bit is also sometimes called as “Modified bit “.
- This bit specifies whether that page has been modified or not.
- If the page has been modified, then this bit is set to 1 otherwise set to 0.

Note: In case the page is modified,

- Before replacing the modified page with some other page, it has to be written back in the secondary memory to avoid losing the data.
- Dirty bit helps to avoid unnecessary writes.
- This is because if the page is not modified, then it can be directly replaced by another page without any need of writing it back to the disk.

**Structure of Page Table** Structure of page table simply defines in how many ways a page table can be structured. Well, the paging is a memory management technique where a large process is divided into pages and is placed in physical memory which is also divided into frames. Frame and page size is equivalent.

There are three common methods that we use to structure a page table.

#### **1. Hierarchical Page Table**

As we knew when the CPU accesses a page of any process it has to be in the main memory. Along with the page, the page table of the same process must also be stored in the main memory. Now, what if the size of the page table is larger than the frame size of the main memory.

In that case, we have to breakdown the page table at multiple levels in order to fit in the frame of the main memory. Let us understand this with the help of an example.

Consider that the size of main memory (physical memory) is 512 MB (i.e. physical memory can be represented with 29 bits)

This physical memory is divided into a number of frames where each frame size = 4 KB = 2<sup>12</sup> (i.e. frame size can be represented with 12 bits)

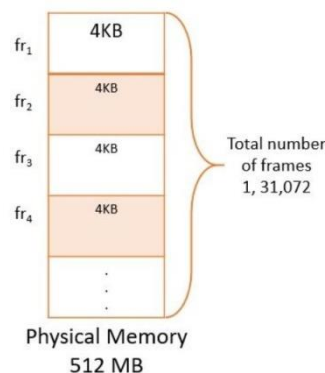
Physical memory in bits = 29 Frame size in bits = 12

Number of bits used to represent frame number = 29 – 12 = 17.

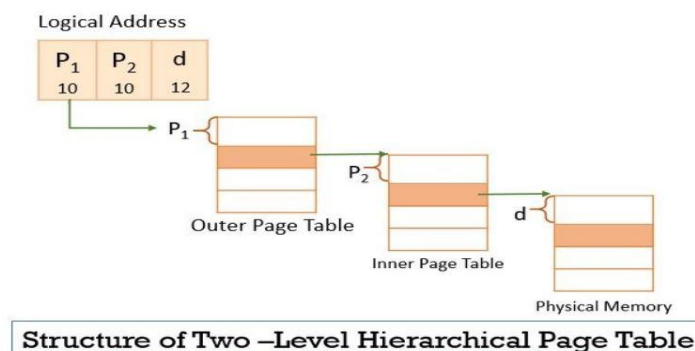
We represent physical memory as:



Total number of frames would be  $2^{17} = 1,31,072$  Now we have a physical memory of 512 MB which is divided into 131072 frames where each frame size is 4 KB.



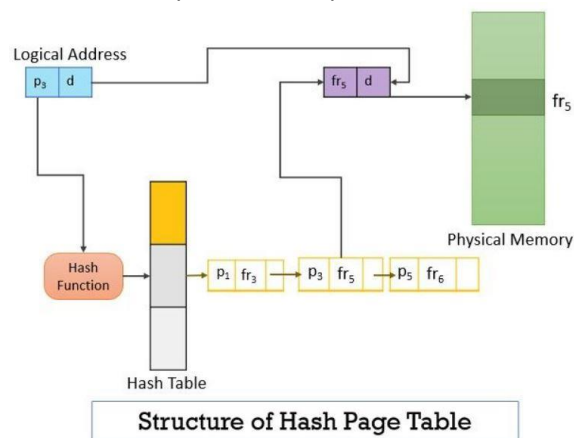
Two-level paging in which a page table itself is paged. So we will have two-page tables' inner page table and outer page table. We have a logical address with page number 20 and page offset 12. As we are paging a page table the page number will further get split to 10-bit page number and 10 bit offset as you can see in the image below.



Here P1 would act as an index and P2 would act as an offset for the outer page table. Further, the P2 would act as an index and d would act as an offset to inner page table to map the logical address of the page to the physical memory.

**2. Hashed Page Table** When the logical address space is beyond 32 bits in such case the page table is structured using hashed page table. Though we can structure the large page table using the multilevel page table it would consist of a number of levels increases the complexity of the page table.

The hashed page table is a convenient way to structure the page table where logical address space is beyond 32 bits. The hash table has several entries where each entry has a link list. Each link list has a set of linked elements where each element hashes to the same location. Each element has three entries page number, frame number and pointer to the next element. We would understand the working of this page table better with the help of an example.



The CPU generates a logical address for the page it needs. Now, this logical address needs to be mapped to the physical address. This logical address has two entries i.e. a page number (P3 in our case) and an offset.

The page number from the logical address is directed to the hash function. The hash function produces a hash value corresponding to the page number. This hash value directs to an entry in the hash table. As we have studied earlier, each entry in the hash table has a link list. Here the page number is compared with the first element's first entry if a match is found then the second entry is checked.

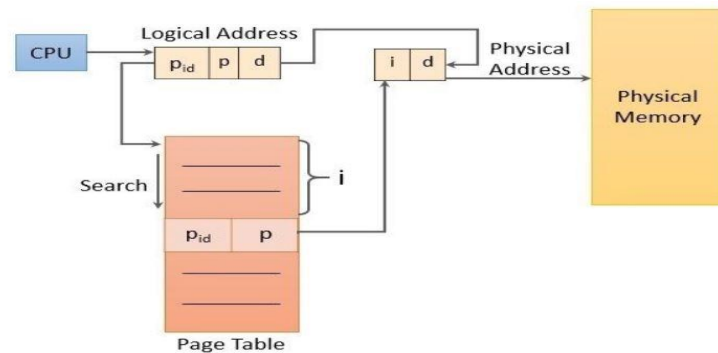
In our example, the logical address includes page number P3 which does not match the first element of the link list as it includes page number P1. So we will move ahead and check the next element; now this element has a page number entry i.e. P3 so further we will check the frame entry of the element which is fr5. To this frame number, we will append the offset provided in the logical address to reach the page's physical address.

So, this is how the hashed page table works to map the logical address to the physical address.

**3. Inverted Page Table** The concept of normal paging says that every process maintains its own page table which includes the entries of all the pages belonging to the process. The large process may have a page table with millions of entries. Such a page table consumes a large amount of memory.

Consider we have six processes in execution. So, six processes will have some or the other of their page in the main memory which would compel their page tables also to be in the main memory consuming a lot of space. This is the drawback of the paging concept.

The inverted page table is the solution to this wastage of memory. The concept of an inverted page table involves the existence of single-page table which has entries of all the pages (May they belong to different processes) in main memory along with the information of the process to which they are associated. To get a better understanding consider the figure below of inverted page table



### Structure of Inverted Page Table

The CPU generates the logical address for the page it needs to access. This time the logical address consists of three entries: process id, page number, and the offset. The process id identifies the process, of which the page has been demanded, page number indicates which page of the process has been asked for, and the offset value indicates the displacement required.

The match of process id along with associated page number is searched in the page table and says if the search is found at the  $i$  entry of page table then  $i$  and offset together generate the physical address for the requested page. This is how the logical address is mapped to a physical address using the inverted page table.

Though the inverted page table reduces the wastage of memory but it increases the search time. This is because the entries in an inverted page table are sorted on the basis of physical address whereas the lookup is performed using logical address. It happens sometimes that the entire table is searched to find the match.

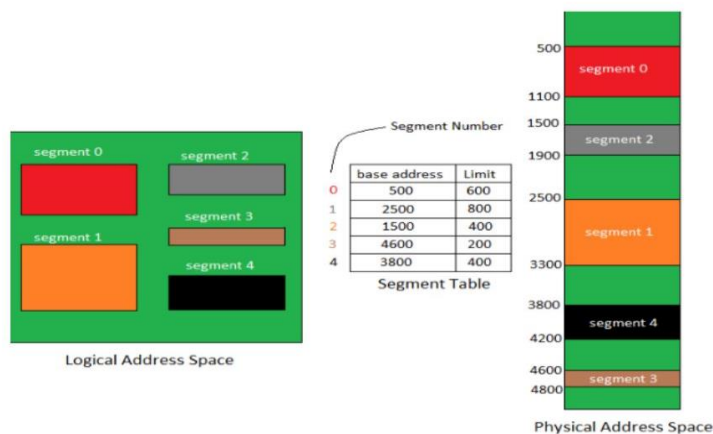
So these are the three techniques that can be used to structure a page table that helps the operating system in mapping the logical address of the page required by CPU to its physical address.

### Assignment: Block Mapping Vs Direct Mapping

#### Segmentation

- Segmentation is a memory management technique in which each job is divided into several segments of different sizes, one for each module that contains pieces that perform related functions. Each segment is actually a different logical address space of the program.

- When a process is to be executed, its corresponding segmentations are loaded into non contiguous memory though every segment is loaded into a contiguous block of available memory.
- Segmentation memory management works very similar to paging but here segments are of variable-length whereas in paging pages are of fixed size.
- A program segment contains the program's main function, utility functions, data structures, and so on. The operating system maintains a segment map table for every process and for each segment; the table stores the starting address of the segment and the length of the segment. A reference to a memory location includes a value that identifies a segment and an offset.



## Paging with Segmentation

- It allows to use segmentation from user's point of view and also to divide each segment into pages of fixed size.
- In a combined paging/segmentation system, a user's address space is broken up into a number of segments. Each segment is broken up into a number of fixed-sized pages which are equal in length to a main memory frame. • Segmentation is visible to the programmer.
- Paging is transparent to the programmer.
- Segments can be of different lengths, so it is harder to find a place for a segment in memory than a page. With segmented virtual memory, we get the benefits of virtual memory but we still have to do dynamic storage allocation of physical memory. In order to avoid this, it is possible to combine segmentation and paging into a two-level virtual memory system. Each segment descriptor points to page table for that segment. This gives some of the advantages of paging (easy placement) with some of the advantages of segments (logical division of the program).

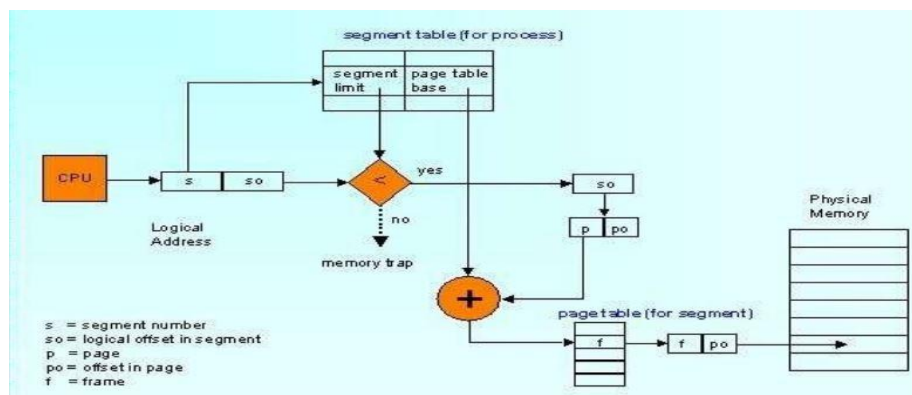


Fig: Address Translation Segmentation with paging

## Advantages

- Reduces memory usage as opposed to pure paging – Page table size limited by segment size – Segment table has only one entry per actual segment
- Share individual pages by copying page table entries.
- Share whole segments by sharing segment table entries, which is the same as sharing the page table for that segment.
- Most advantages of paging still hold – Simplifies memory allocation – Eliminates external fragmentation.
- In general, this system combines the efficiency in paging with the protection and sharing capabilities of the segmentation

## Virtual memory

- Virtual memory is a memory management technique that allows programs larger than physical memory to be executed.
- A computer can address more memory than the amount physically installed on the system. This extra memory is actually called virtual memory.
- The run time mapping from virtual address to physical address is done by hardware device called memory management unit (MMU).
- Virtual Memory is a space where large programs can store themselves in form of pages while their execution and only the required pages or portions of processes are loaded into the main memory.
- Virtual memory takes the advantage of the fact that processes typically do not use all the allocated memory. In other words, most processes never need all their pages at once.

## Demand paging and pre-paging

With demand paging, a page is brought into memory only when a location on that page is actually referenced during execution. With pre-paging, pages other than the one demanded by a page fault are brought in. The selection of such pages is done based on common access patterns, especially for secondary memory devices.

## Page Fault

- When the page (data) requested by a program is not available in the memory, it is called as a page fault.

| Page reference stream: |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |
|------------------------|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 1                      | 2 | 3 | 2 | 1 | 5 | 2 | 1 | 6 | 2 | 5 | 6 | 3 | 1 | 3 | 6 | 1 | 2 | 4 | 3 |
| 1                      | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 6 | 6 | 6 | 6 | 6 | 6 | 6 | 6 | 6 | 2 | 2 | 2 |
|                        | 2 | 2 | 2 | 2 | 2 | 2 | 2 | 2 | 2 | 2 | 2 | 2 | 2 | 1 | 1 | 1 | 1 | 4 | 4 |
|                        |   | 3 | 3 | 3 | 5 | 5 | 5 | 5 | 5 | 5 | 5 | 5 | 3 | 3 | 3 | 3 | 3 | 3 | 3 |
| *                      | * | * |   |   | * |   |   | * |   |   |   |   | * | * |   |   | * | * |   |
| Optimal                |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |
| Total 9 page faults    |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |

### Least Recently Used (LRU) algorithm

- Page which has not been used for the longest time in main memory is the one which will be selected for replacement.
- Easy to implement, keep a list, replace pages by looking back into time.

Page reference stream:

|   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 1 | 2 | 3 | 2 | 1 | 5 | 2 | 1 | 6 | 2 | 5 | 6 | 3 | 1 | 3 | 6 | 1 | 2 | 4 | 3 |
| 1 | 1 | 1 | 1 | 3 | 2 | 1 | 5 | 2 | 1 | 6 | 2 | 5 | 6 | 6 | 1 | 3 | 6 | 1 | 2 |
|   | 2 | 2 | 3 | 2 | 1 | 5 | 2 | 1 | 6 | 2 | 5 | 6 | 3 | 1 | 3 | 6 | 1 | 2 | 4 |
|   |   | 3 | 2 | 1 | 5 | 2 | 1 | 6 | 2 | 5 | 6 | 3 | 1 | 3 | 6 | 1 | 2 | 4 | 3 |
| * | * | * |   |   | * |   |   | * |   | * |   | * | * |   |   | * | * | * |   |

LRU

Total 11 page faults

### The Not Recently Used

The not recently used (NRU) page replacement algorithm is an algorithm that favors keeping pages in memory that have been recently used. This algorithm works on the following principle: when a page is referenced, a referenced bit is set for that page, marking it as referenced. Similarly, when a page is modified (written to), a modified bit is set. The setting of the bits is usually done by the hardware, although it is possible to do so on the software level as well. At a certain fixed-time interval, a timer interrupt triggers and clears the referenced bit of all the pages, so only pages referenced within the current timer interval are marked with a referenced bit. When a page needs to be replaced, the operating system divides the pages into four classes:

3. referenced, modified
2. referenced, not modified
1. Not referenced, modified
0. Not referenced, not modified

Although it does not seem possible for a page to be modified yet not referenced, this happens when a class 3 page has its referenced bit cleared by the timer interrupt. The NRU algorithm picks a random page from the lowest category for removal. So out of the above four-page categories, the NRU algorithm will replace a not-referenced, not-modified page if such a page exists. Note that this algorithm implies that a modified but not-referenced (within the last timer interval) page is less important than a not-modified page that is intensely referenced.

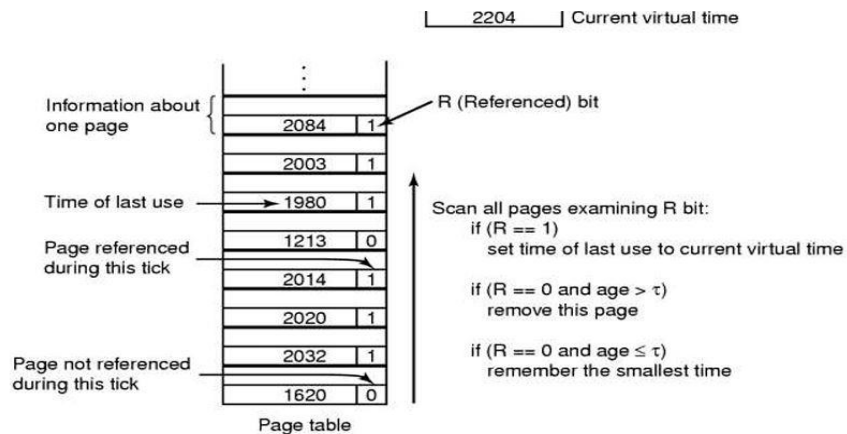
### Clock Page Replacement Algorithm

Clock is a more efficient version of FIFO than Second-chance because pages don't have to be constantly pushed to the back of the list, but it performs the same general function as Second-Chance. The clock algorithm keeps a circular list of pages in memory, with the "hand" (iterate) pointing to the last examined page frame in the list. When a page fault occurs and no empty frames exist, then the R (referenced) bit is inspected at the hand's location. If R is 0, the new page is put in place of the page the

If  $R$  is 0, the page has not been referenced during the current clock tick and may be a candidate for removal. To see whether or not it should be removed, its age, that is, the current virtual time minus its

Time of last use is computed and compared to  $t$ . If the age is greater than  $t$ , the page is no longer in the working set. It is reclaimed and the new page loaded here. The scan continues updating the remaining entries, however.

However, if  $R$  is 0 but the age is less than or equal to  $t$ , the page is still in the working set. The page is temporarily spared, but the page with the greatest age (smallest value of Time of last use) is noted. If the entire table is scanned without finding a candidate to evict, that means that all pages are in the working set. In that case, if one or more pages with  $R = 0$  were found, the one with the greatest age is evicted. In the worst case, all pages have been referenced during the current clock tick (and thus all have  $R = 1$ ), so one is chosen at random for removal, preferably a clean page, if one exists.



## Thrashing

Thrashing is term referred to a high paging activity. A process is thrashing if it is spending more of its time paging than actually executing. Consider a process that currently needs some more frames. Currently all the frames are occupied. So, it will have to swap with certain pages; however, since all the pages are actively being used, there would be need for an immediate page replacement after this. Such a scenario is called thrashing. It results in serious performance problems.



## File

A file is sequence of logical records i.e. a sequence of bits and bytes. A file is a named collection of related information that is recorded on secondary storage such as magnetic disks, magnetic tapes and optical disks. In general, a file is a sequence of bits, bytes, lines or records whose meaning is defined by the file's creator and user.

## **File System**

A file system is the methods and data structures that an operating system uses to keep track of files on a disk or partition; that is, the way the files are organized on the disk. The word is also used to refer to a partition or disk that is used to store the files or the type of the file system.

## **File Attributes**

A file has a name and data. Other than this, it contains information like date and time of file creation, file's current size etc. with a file. These are known as attributes of a file. The lists of file attributes are:

- Name –only information kept in human-readable form
- Identifier–unique tag (number) identifies file within file system
- Type–needed for systems that support different types
- Location–This information is a pointer to a device and to the location of the file on that device
- Size–current file size and possibly the maximum allowed size
- Protection–Access-control information determines who can do reading, writing, executing
- Time, date, and user identification –data for protection, security, and usage monitoring etc.

## **File Naming**

When a process creates a file, it gives the file a name. When the process terminates, the file continues to exist and can be accessed by other processes using its name.

File naming = File name + File extension

Eg. bcabit.doc In this example the bcabit is the file name and doc is the file extension.

Some common file extensions are: DOC and DOCX, .HTML and .HTM,.ODT,.PDF,.XLS and XLSX, .ODS, .PPT and .PPTX,.TXT etc.

## **File Operation**

CREATE: A blank file is created.

DELETE: The purpose of this system call is to delete this file

OPEN: Open either a file for reading or writing.

CLOSE: When a file is no longer accessed. READ: When a file is only to be read.

WRITE: To write some data on file.

Seek: Operation allows the application program to change the value of the file pointer so that subsequent Read/Write is performed from a new position in the file. The new value of the file pointer is determined by adding the offset to the current value.

APPEND: To add some data to the end of the file.

RENAME: To rename the file.

## **File Type**

File type refers to the ability of the operating system to distinguish different types of files such as text files, source files and binary files etc. Many operating systems support many types of files. Operating system like MS-DOS and UNIX have the following types of files –

### **1. Ordinary files (Regular files)**

These are the files that contain user information. These may have text, databases or executable program. The user can apply various operations on such files like add, modify, delete or even remove the entire file.

### **2. Directory files**

These files contain list of file names and other information related to these files.

### **3. Special files (Device files)**

These files are also known as device files. These files represent physical device like disks, terminals, printers, network, tape drive etc. These files are of two types –

- Character special files – data is handled character by character as in case of terminals or printers.
- Block special files – data is handled in blocks as in the case of disks and tapes.

## **File Access Mechanisms**

File access mechanism refers to the manner in which the records of a file may be accessed. There are several ways to access files –

- Sequential access
- Direct/Random access
- Indexed sequential access
- 

### **Sequential access**

A sequential access is that in which the records are accessed in some sequence, i.e., the information in the file is processed in order, one record after the other.

This access method is the most primitive one.

Example: Compilers usually access files in this fashion.

### **Direct/Random access**

Random access file organization provides, accessing the records directly.

Each record has its own address on the file by the help of which it can be directly accessed for reading or writing.

The records need not be in any sequence within the file and they need not be in adjacent locations on the storage medium.

Example: Accessing file in hard disk.

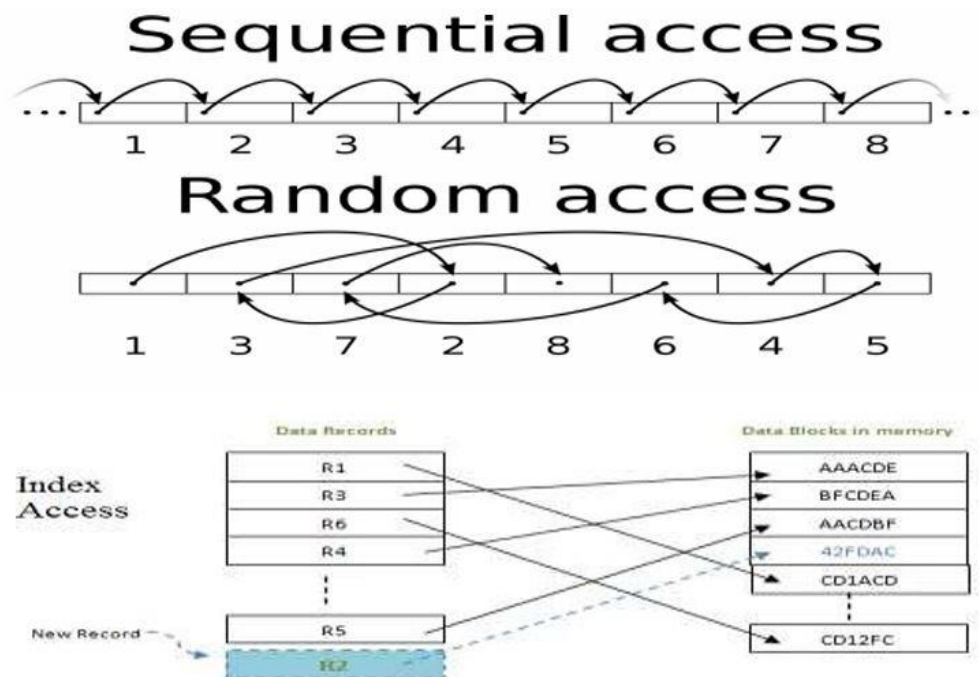
### Indexed sequential access

This mechanism is built up on base of sequential access.

An index is created for each file which contains pointers to various blocks.

Index is searched sequentially and its pointer is used to access the file directly.

Example: Accessing files in payroll system.



## Directories

Directory is the collection of files. The directory contains information about the files, including attributes, location and ownership. Much of this information especially that is concerned with storage is managed by the operating system.

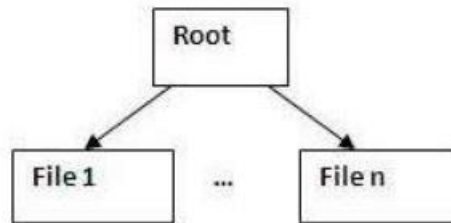
### Types of directories

A directory system can be classified into single level and hierarchical directory system:

#### i. Single level directory system:

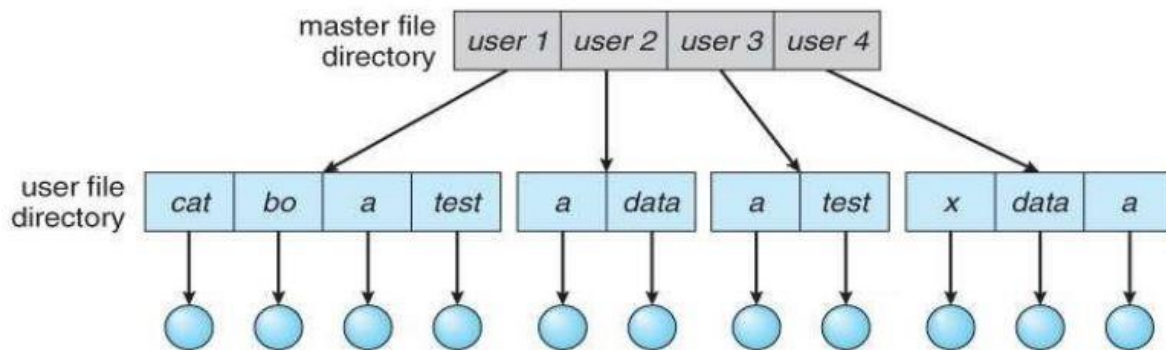
- In this type of directory system, there is a root directory which has all files.
- It has a simple architecture and there are no sub directories.
- Advantage of single level directory system is that it is easy to find a file in the directory.

- This type of directory system is used in cameras and phones.



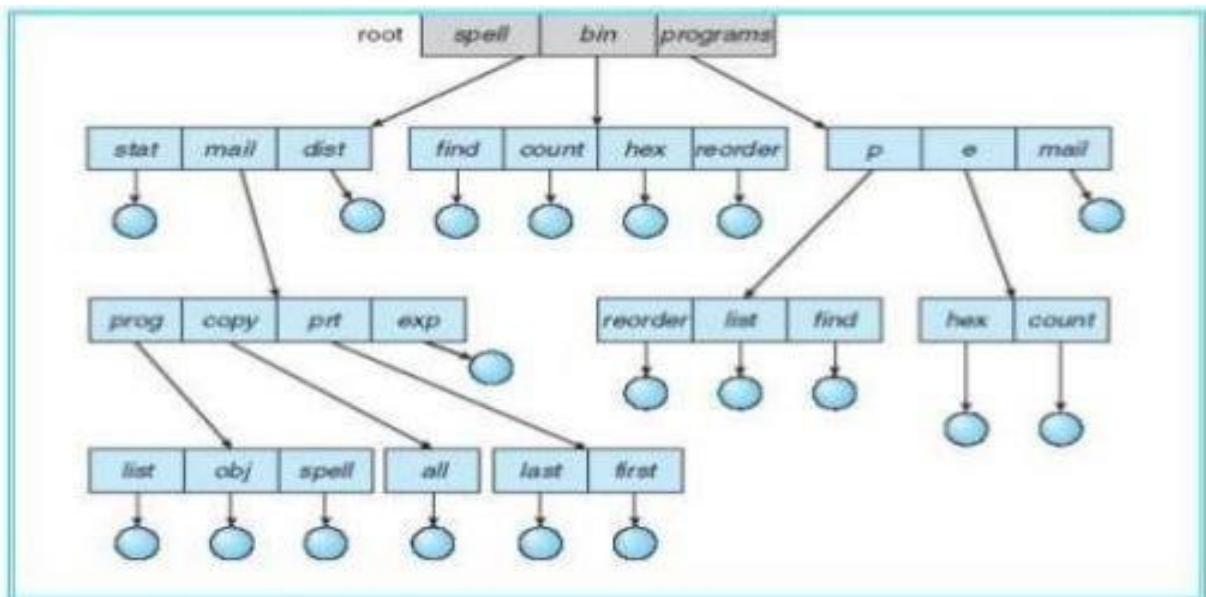
## ii. Two-Level Directory

- Each user gets their own directory space.
  - File names only need to be unique within a given user's directory.
  - A master file directory is used to keep track of each user's directory, and must be maintained when users are added to or removed from the system.
  - A separate directory is generally needed for system (executable) files.
  - Systems may or may not allow users to access other directories besides their own
- If access to other directories is allowed, then provision must be made to specify the directory being accessed.
- If access is denied, then special consideration must be made for users to run programs located in system directories.



## iii. Hierarchical directory or Tree directory

- In a hierarchical directory system, files are grouped together to form a sub directory.
- At the top of the hierarchy is the root directory and then there are sub directories which has files.
- Advantage of hierarchical directory system is that users can be provided access to a sub directory rather than the entire directory. It provides a better structure to file system. Also, managing millions of files is easy with this architecture.
- Personal computers use hierarchical directory system for managing files



## Protection Mechanism

Protection mechanism refers to any mechanism for controlling access of processes or users to resources defined by the operating system. Several strategies for Protection mechanism are:

### Authentication

Authentication refers to identifying each user of the system and associating the executing programs with those users. It is the responsibility of the Operating System to create a protection system which ensures that a user who is running a particular program is authentic. Operating Systems generally identifies/authenticates users using following three ways –

- **Username / Password** – User need to enter a registered username and password with Operating system to login into the system.
- **User card/key** – User need to use punch card in card slot, or enter key generated by key generator in option provided by operating system to login into the system.
- **User attribute - fingerprint/ eye retina pattern/ signature** – User need to pass his/her attribute via designated input device used by operating system to login into the system.

### One Time passwords

One-time passwords provide additional security along with normal authentication. In One-Time Password system, a unique password is required every time user tries to login into the system. Once a one-time password is used, then it cannot be used again. One-time passwords are implemented in various ways.

- **Random numbers** – Users are provided cards having numbers printed along with corresponding alphabets. System asks for numbers corresponding to few alphabets randomly chosen.
- **Secret key** – Users are provided a hardware device which can create a secret id mapped with user id. System asks for such secret id which is to be generated every time prior to login.

- **Network password** – Some commercial applications send one-time passwords to user on registered mobile/ email which is required to be entered prior to login.

## Encryption

In computing, encryption is the method by which plaintext or any other type of data is converted from a readable form to an encoded version that can only be decoded by another entity if they have access to a decryption key. Encryption is one of the most important methods for providing data security, especially for end-to-end protection of data transmitted across networks.

Encryption is widely used on the internet to protect user information being sent between a browser and a server, including passwords, payment information and other personal information that should be considered private. Organizations and individuals also commonly use encryption to protect sensitive data stored on computers, servers and mobile devices like phones or tablets.

Unencrypted data, often referred to as plaintext, is encrypted using an encryption algorithm and an encryption key. This process generates cipher text that can only be viewed in its original form if decrypted with the correct key. Decryption is simply the inverse of encryption, following the same steps but reversing the order in which the keys are applied.

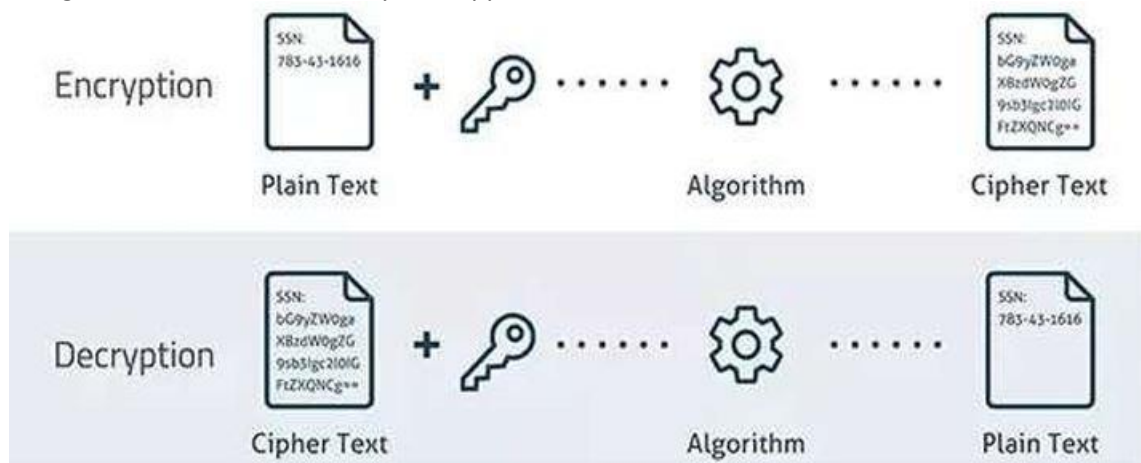


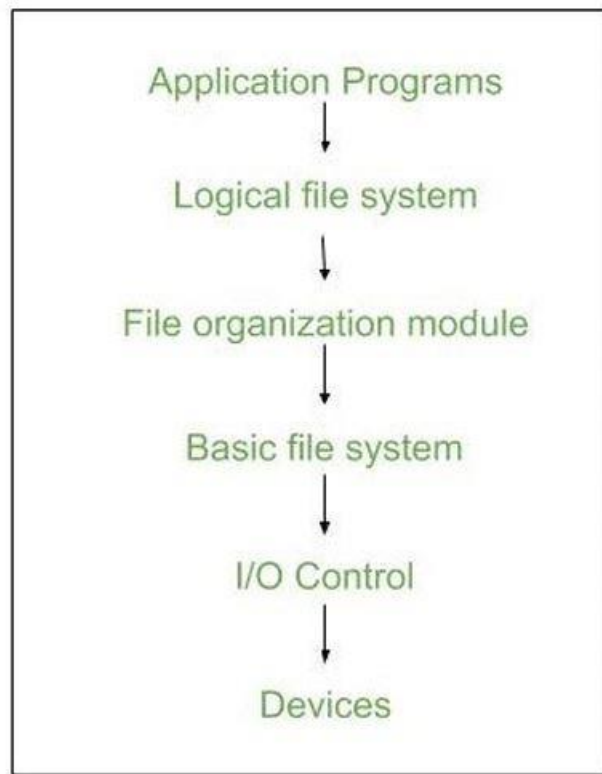
Fig: Encryption and decryption process

## File System Structure

File System provide efficient access to the disk by allowing data to be stored, located and retrieved in a convenient way. A file System must be able to store the file, locate the file and retrieve the file.

Most of the Operating Systems use layering approach for every task including file systems. Every layer of the file system is responsible for some activities.

The image shown below, elaborates how the file system is divided in different layers, and also the functionality of each layer.



When an application program asks for a file, the first request is directed to the logical file system. The logical file system contains the Meta data of the file and directory structure. If the application program doesn't have the required permissions of the file, then this layer will throw an error. Logical file systems also verify the path to the file.

Generally, files are divided into various logical blocks. Files are to be stored in the hard disk and to be retrieved from the hard disk. Hard disk is divided into various tracks and sectors. Therefore, in order to store and retrieve the files, the logical blocks need to be mapped to physical blocks. This mapping is done by File organization module. It is also responsible for free space management.

Once File organization module decided which physical block the application program needs, it passes this information to basic file system. The basic file system is responsible for issuing the commands to I/O control in order to fetch those blocks.

I/O controls contain the codes by using which it can access hard disk. These codes are known as device drivers. I/O controls are also responsible for handling interrupts.

### **Methods of allocation**

The allocation methods define how the files are stored in the disk blocks. There are three main disk space or file allocation methods.

- i. Contiguous Allocation
- ii. Linked Allocation
- iii. Indexed Allocation

The main idea behind these methods is to provide:

- Efficient disk space utilization.
- Fast access to the file blocks

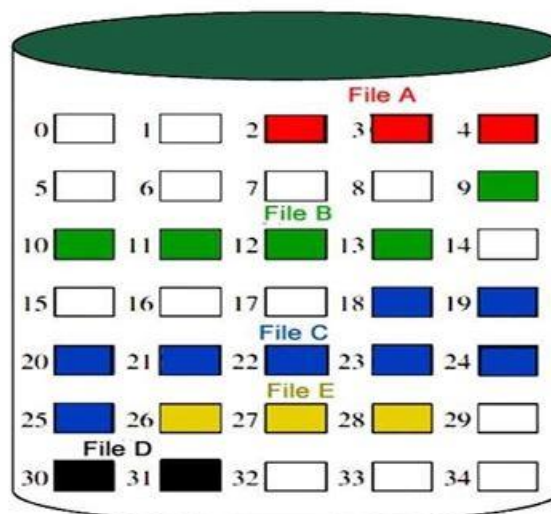
## 1. Contiguous Allocation

In this scheme, each file occupies a contiguous set of blocks on the disk. For example, if a file requires  $n$  blocks and is given a block  $b$  as the starting location, then the blocks assigned to the file will be:  $b, b+1, b+2, \dots, b+n-1$ . This means that given the starting block address and the length of the file (in terms of blocks required), we can determine the blocks occupied by the file.

For example, If we have a disk with 1 KB blocks, then for storing a 50 KB file, we will require 50 consecutive blocks.

The file allocation table in contiguous allocation contains

- Address of starting block
- Length of the allocated portion. The file 'File A' in the above figure starts from the block 2 with length = 3 blocks. Therefore, it occupies 2,3 and 4 blocks.



File allocation table

| File name | Start block | Length |
|-----------|-------------|--------|
| File A    | 2           | 3      |
| File B    | 9           | 5      |
| File C    | 18          | 8      |
| File D    | 30          | 2      |
| File E    | 26          | 3      |

### Advantages:

- Both the Sequential and Direct Accesses are supported by this. For direct access, the address of the  $k$ th block of the file which starts at block  $b$  can easily be obtained as  $(b+k)$ .

- This is extremely fast since the number of seeks are minimal because of contiguous allocation of file blocks.

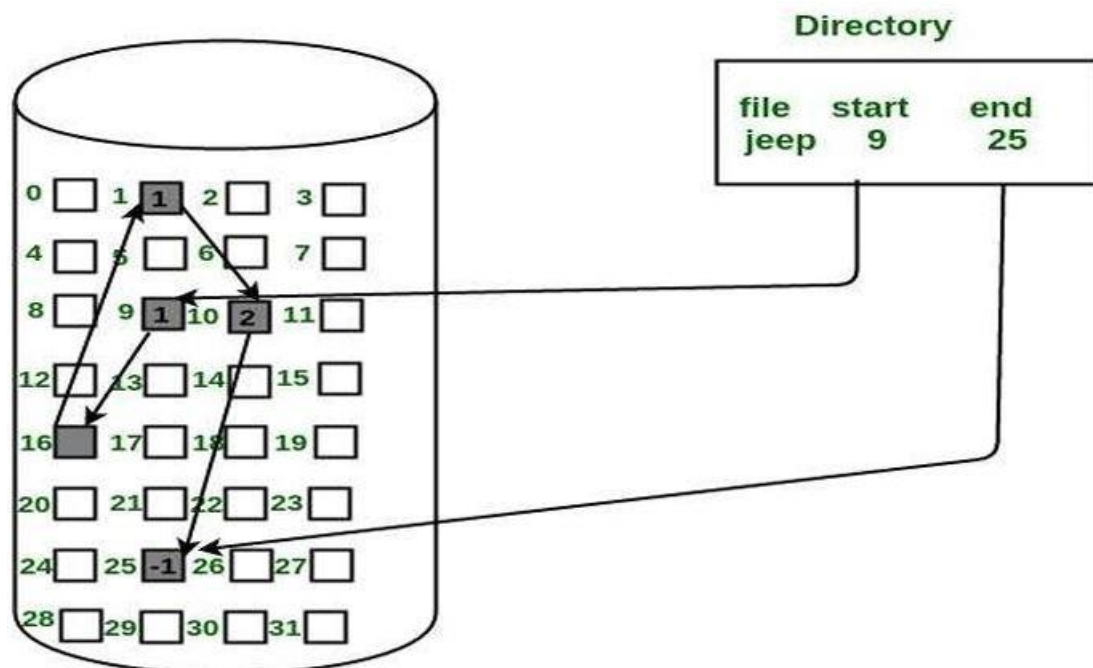
#### Disadvantages:

- This method suffers from internal and external fragmentation. This makes it inefficient in terms of memory utilization.
- Increasing file size is difficult because it depends on the availability of contiguous memory at a particular instance.

## 2. Linked Allocation (Non-contiguous allocation)

In this scheme, each file is a linked list of disk blocks which need not be contiguous. The disk blocks can be scattered anywhere on the disk. The file allocation table contains a pointer to the starting and the ending file block. Each block contains a pointer to the next block occupied by the file.

The file 'jeep' in following image shows how the blocks are randomly distributed. The last block (25) contains -1 indicating a null pointer and does not point to any other block.



#### Advantages:

- This is very flexible in terms of file size. File size can be increased easily since the system does not have to look for a contiguous chunk of memory.
- This method does not suffer from external fragmentation. This makes it relatively better in terms of memory utilization.

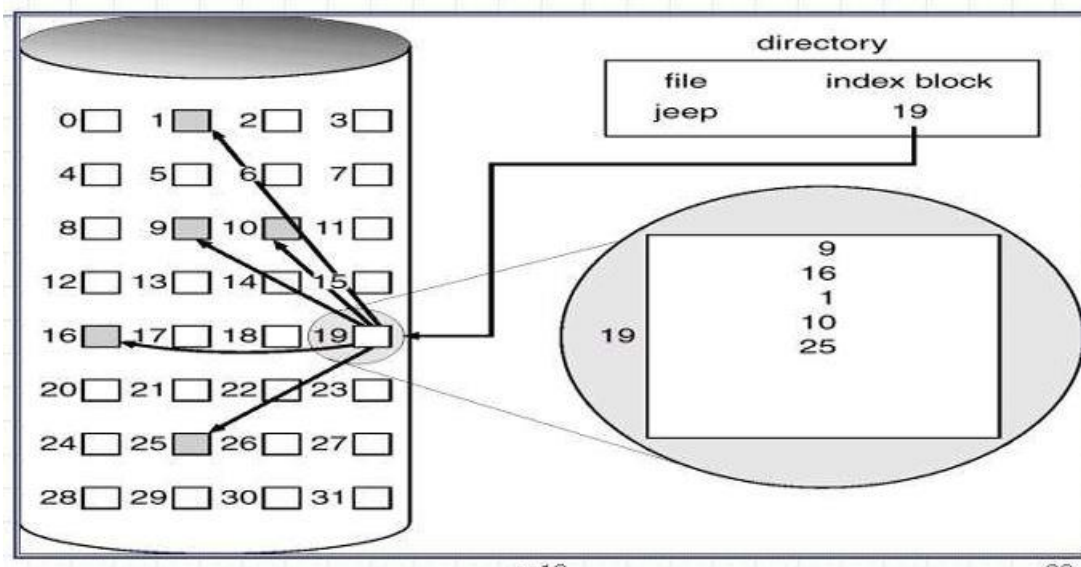
#### Disadvantages:

Because the file blocks are distributed randomly on the disk, a large number of seeks are needed to access every block individually. This makes linked allocation slower.

- It does not support random or direct access. We cannot directly access the blocks of a file. A block  $k$  of a file can be accessed by traversing  $k$  blocks sequentially (sequential access) from the starting block of the file via block pointers.
- Pointers required in the linked allocation incur some extra overhead.

### 3. Indexed Allocation (i-nodes)

In this scheme, a special block known as the Index block contains the pointers to all the blocks occupied by a file. Each file has its own index block. The  $i$ th entry in the index block contains the disk address of the  $i$ th file block. The directory entry contains the address of the index block as shown below:



#### Advantages:

- This supports direct access to the blocks occupied by the file and therefore provides fast access to the file blocks.
- It overcomes the problem of external fragmentation.

#### Disadvantages:

- The pointer overhead for indexed allocation is greater than linked allocation.
- For very small files, say files that expand only 2-3 blocks, the indexed allocation would keep one entire block (index block) for the pointers which is inefficient in terms of memory utilization. However, in linked allocation we lose the space of only 1 pointer per block.

## Free space management

The system maintains a free space list by keep track of the free disk space. The free space list contains all the records of the free space disk block. The free blocks are those which are not allocated to another file or directory. When we create a file, we first search for the free space in the memory and then check in the free space list for the required amount of space that we require for our file. If the free space is available then allocate this space to the new file. After that, the allocating space is deleted from the free space list. Whenever we delete a file then its free memory space is added to the free space list.

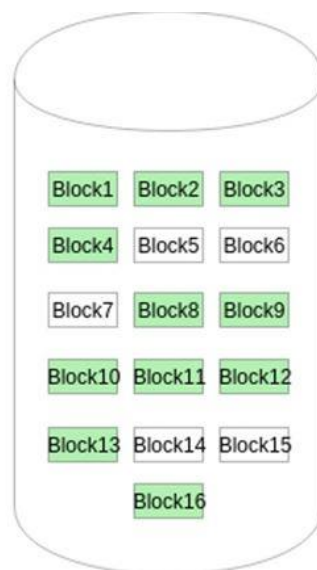
The process of looking after and managing the free blocks of the disk is called free space management. There are some methods or techniques to implement a free space list. These are as follows:

- Bitmap
- Linked list
- Grouping
- Counting

### • Bitmap or Bit vector –

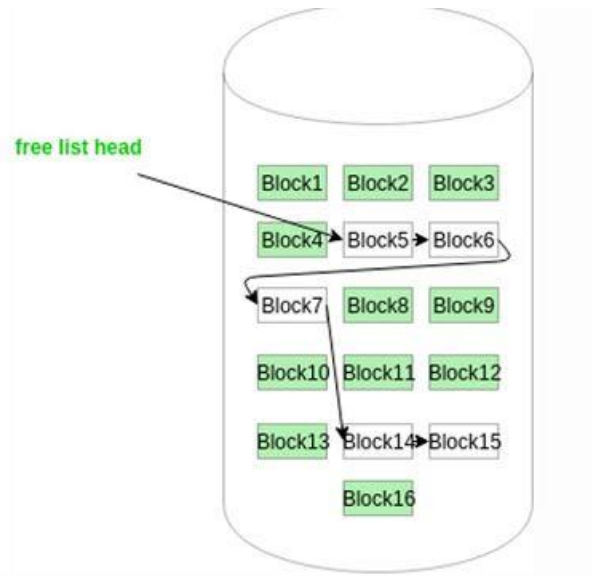
A Bitmap or Bit Vector is series or collection of bits where each bit corresponds to a disk block. The bit can take two values: 0 and 1: 0 indicates that the block is allocated and 1 indicates a free block. The given instance of disk blocks on the disk in Figure 1 (where green blocks are allocated) can be represented by a bitmap of 16 bits as: 0000111000000110.

- The block number can be calculated as:  $(\text{number of bits per word}) * (\text{number of 0-values words}) + \text{offset of bit first bit 1 in the non-zero word}$ .
- we scan the bitmap sequentially for the first non-zero word. The first group of 8 bits (00001110) constitute a non-zero word since all bits are not 0. After the non-0 word is found, we look for the first 1 bit. This is the 5th bit of the non zero word. So, offset = 5. Therefore, the first free block number =  $8*0+5 = 5$ .



## 1. Linked List –

In this approach, the free disk blocks are linked together i.e. a free block contains a pointer to the next free block. The block number of the very first disk block is stored at a separate location on disk and is also cached in memory.



In Figure-above, the free space list head points to Block 5 which points to Block 6, the next free block and so on. The last free block would contain a null pointer indicating the end of free list.

A drawback of this method is the I/O required for free space list traversal.

## 2. Grouping –

This approach stores the address of the free blocks in the first free block. The first free block stores the address of some, say  $n$  free blocks. Out of these  $n$  blocks, the first  $n-1$  blocks are actually free and the last block contains the address of next free  $n$  blocks.

An advantage of this approach is that the addresses of a group of free disk blocks can be found easily.

## 3. Counting –

This approach stores the address of the first free disk block and a number  $n$  of free contiguous disk blocks that follow the first block. Every entry in the list would contain:

1. Address of first free disk block
2. A number  $n$  For example, in Figure-1, the first entry of the free space list would be: ([Address of Block 5], 2), because 2 contiguous free blocks follow block 5.

## Directory Implementation

There is the number of algorithms by using which, the directories can be implemented. However, the selection of an appropriate directory implementation algorithm may significantly affect the performance of the system.

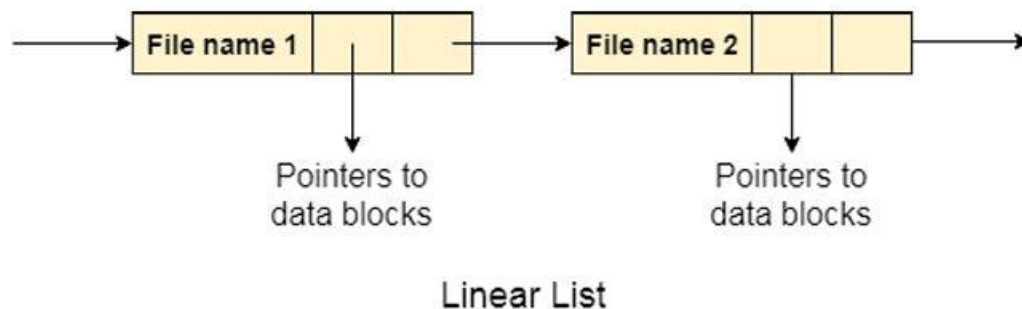
The directory implementation algorithms are classified according to the data structure they are using. There are mainly two algorithms which are used in these days.

### 1. Linear List

In this algorithm, all the files in a directory are maintained as singly lined list. Each file contains the pointers to the data blocks which are assigned to it and the next file in the directory.

#### Characteristics

1. When a new file is created, then the entire list is checked whether the new file name is matching to a existing file name or not. In case, it doesn't exist, the file can be created at the beginning or at the end. Therefore, searching for a unique name is a big concern because traversing the whole list takes time.
2. The list needs to be traversed in case of every operation (creation, deletion, updating, etc) on the files therefore the systems become inefficient.



### 2. Hash Table

To overcome the drawbacks of singly linked list implementation of directories, there is an alternative approach that is hash table. This approach suggests to use hash table along with the linked lists.

A key-value pair for each file in the directory gets generated and stored in the hash table. The key can be determined by applying the hash function on the file name while the key points to the corresponding file stored in the directory.

Now, searching becomes efficient due to the fact that now, entire list will not be searched on every operating. Only hash table entries are checked using the key and if an entry found then the corresponding file will be fetched using the value.

